

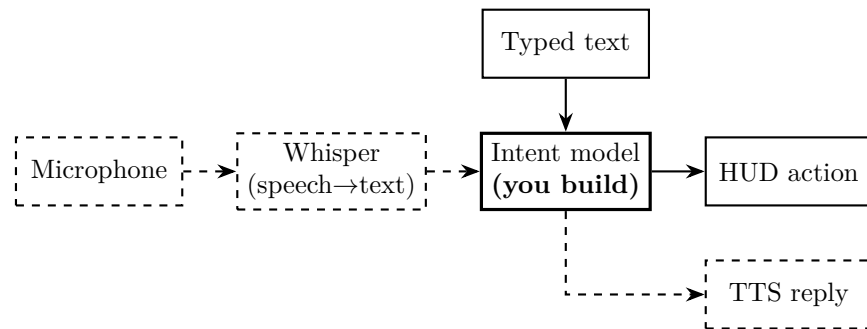
CLAWS Onboarding: EVA Voice Intent Model

CLAWS Voice Assistant Team

Fall 2026

Goal: write training sentences for 36 EVA intents, fine-tune a small model, classify sentences it has never seen. Text in, intent out is required. Voice (Whisper + TTS) is a stretch goal. Shared code is in `code/`; your work goes in your own folder.

1 Where this fits



Solid = required. Dashed = stretch. The model picks an intent (plus a name for five intents), or **unhandled** if the speech isn't a command. The HUD reads the actual value from telemetry.

Reviewed and merged by: Ayush.

2 The intents

Labels are **case-sensitive**. Copy them exactly. Also in `code/intents.json` and a copy in your folder. `unhandled` is a real intent: write 30+ things an astronaut might say that aren't commands ("nice view", "copy that", "hold on").

Intent	Meaning	Name
<code>get_oxy_pri_storage</code>	Primary oxygen storage %	
<code>get_oxy_sec_storage</code>	Secondary oxygen storage %	
<code>get_oxy_pri_pressure</code>	Primary oxygen tank pressure	
<code>get_oxy_sec_pressure</code>	Secondary oxygen tank pressure	
<code>get_suit_pressure_oxy</code>	Suit O ₂ partial pressure	
<code>get_suit_pressure_co2</code>	Suit CO ₂ partial pressure	
<code>get_suit_pressure_other</code>	Suit trace gas pressure	
<code>get_suit_pressure_total</code>	Total suit pressure	
<code>get_helmet_pressure_co2</code>	Helmet CO ₂ pressure	
<code>get_fan_pri_rpm</code>	Primary fan speed (RPM)	
<code>get_fan_sec_rpm</code>	Secondary fan speed (RPM)	
<code>get_scrubber_a_co2_storage</code>	Scrubber A CO ₂ storage	
<code>get_scrubber_b_co2_storage</code>	Scrubber B CO ₂ storage	
<code>get_temperature</code>	Suit internal temperature	
<code>get_coolant_storage</code>	Coolant storage %	
<code>get_coolant_gas_pressure</code>	Coolant gas loop pressure	
<code>get_coolant_liquid_pressure</code>	Coolant liquid loop pressure	
<code>get_heart_rate</code>	Astronaut heart rate	
<code>get_oxy_consumption</code>	Oxygen consumption rate	
<code>get_co2_production</code>	CO ₂ production rate	
<code>get_eva_elapsed_time</code>	Elapsed mission time	
<code>get_primary_battery_level</code>	Primary battery %	
<code>get_secondary_battery_level</code>	Secondary battery %	
<code>get_warnings</code>	Active warnings and alerts	
<code>open_menu_vitals</code>	Open vitals panel	
<code>open_menu_navigation</code>	Open navigation / map panel	
<code>open_menu_tasks</code>	Open tasks / checklist panel	
<code>open_menu_uia</code>	Open UIA airlock panel	
<code>close_menu</code>	Close the active panel	
<code>Get_coordinates</code>	Coordinates of a named target	✓
<code>Set_navigation_target</code>	Navigate to a named place	✓
<code>reroute_navigation</code>	Compute a new route	
<code>Add_waypoint</code>	Add a named waypoint	✓
<code>Delete_waypoint</code>	Delete a named waypoint	✓
<code>Complete_task</code>	Mark a named task complete	✓
<code>unhandled</code>	Not a command: small talk, noise, off-topic	

3 Setup

Any laptop works (CPU is fine; MPS/CUDA used if present). Use **Python 3.10–3.12** (newer versions may lack PyTorch builds). Your folder is already made:

Member	Folder
Aaron	submissions/aaron/
Evan	submissions/evan/
Gloria	submissions/gloria/
Raiana	submissions/raiana/
Sam	submissions/sam/
Utsav	submissions/utsav/
Vivian	submissions/vivian/

```
git clone https://github.com/CLAWS-UMICH/AI-Onboarding-2026.git
cd AI-Onboarding-2026
git checkout -b onboarding/<your-name>
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cd submissions/<your-name>           # run everything from here
```

Windows (PowerShell)

```
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
cd submissions\<your-name>
```

If PowerShell refuses to run `Activate.ps1`, run this once:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

In Command Prompt use `.venv\Scripts\activate.bat`. After activating, `python` works the same on every OS and all commands below use forward slashes, which Windows accepts.

Each folder has an empty `data/train.jsonl`, an empty `runs/`, a `notes.md` template, `MODELS.md` (the model options below) and a copy of `intents.json`. Run scripts from your folder as `python ../../code/<script>.py`; data and results land in your folder. Never edit `code/`. Your PR should only touch your own folder.

4 Write the training data

The model is only as good as your sentences. One file, `data/train.jsonl`, one JSON object per line, exactly one intent each:

```
{ "text": "what's my primary oxygen at", "intents": ["get_oxy_pri_storage"] }
{ "text": "O2 secondary tank pressure?", "intents": ["get_oxy_sec_pressure"] }
{ "text": "uh how hot is it in the suit", "intents": ["get_temperature"] }
{ "text": "pull up the map", "intents": ["open_menu_navigation"] }
{ "text": "drop a waypoint called Crater Rim", "intents": ["Add_waypoint"] }
{ "text": "copy that, heading out", "intents": ["unhandled"] }
{ "text": "whats the heart rate", "intents": ["get_heart_rate"], "source": "llm" }
```

Rules

- **30+ per intent** (50 ideal). Write them by hand, generate them synthetically (e.g. with an LLM), or mix both. Check every generated row and tag it `"source": "llm"`. The validator counts both.
- Save the file as UTF-8 (VS Code does by default).
- Mix questions, commands and clipped phrases (“fan RPM?”). 2 to 20 words.
- Vary words: oxygen/O2/ox, battery/charge/power, menu/panel/screen.
- Sound like speech: “uh”, “please”, lowercase, no punctuation.
- Always say which one: primary/secondary, A/B, gas/liquid.
- Vary names in the five name intents. No near-duplicates.

Listing 1: `validate.py`

```
"""Check a training file: valid JSON, known labels, one label per row, no duplicates,
    enough rows."""
import json
import sys
from collections import Counter
from pathlib import Path

MIN_PER_INTENT = 30    # rows per intent; handwritten or synthetic both count

if len(sys.argv) != 2:
    sys.exit("usage: python validate.py data/train.jsonl")

intents_file = Path(__file__).with_name("intents.json")
labels = {x["intent"] for x in json.loads(intents_file.read_text(encoding="utf-8"))}
seen, total, problems = set(), Counter(), []

for n, line in enumerate(open(sys.argv[1], encoding="utf-8"), 1):
    if not line.strip():
        continue
    try:
        row = json.loads(line)
        text, intents = row["text"].strip(), row["intents"]
    except (ValueError, KeyError, TypeError, AttributeError):
```

```

        problems.append(f'line {n}: not a valid row, expected {{"text": "...", "intents":
["label"]}}}')
        continue
    if not isinstance(intents, list) or len(intents) != 1 or intents[0] not in labels:
        problems.append(f"line {n}: bad intents {intents!r} (need exactly one label from
intents.json)")
        continue
    if text.lower() in seen:
        problems.append(f"line {n}: duplicate text '{text}'")
    seen.add(text.lower())
    total[intents[0]] += 1

for label in sorted(labels):
    if total[label] < MIN_PER_INTENT:
        problems.append(f"{label}: only {total[label]} rows (need {MIN_PER_INTENT})")

print("\n".join(problems))
print(f"{sum(total.values())} rows, {len(problems)} problems")
sys.exit(1 if problems else 0)

```

```
python ../../code/validate.py data/train.jsonl    # fix until "0 problems"
```

5 Build the model

Pick an encoder. Three options, all on Hugging Face. MiniLM is the default; try another once your data is solid. Click a name for its model page (huggingface.co/<model>).

Model	Params	Hidden	Notes
sentence-transformers/all-MiniLM-L6-v2	22M	384	default, fastest
BAAI/bge-small-en-v1.5	33M	384	often more accurate
sentence-transformers/all-mpnet-base-v2	110M	768	biggest, $\sim 5\times$ slower

Set `INTENT_MODEL` before running the baseline and training. Both use it, so you compare like with like (the baseline pools each model the way it was published; fine-tuning always mean-pools). `predict.py` reads your choice back from `runs/metrics.json`. Changed models? Re-run both.

```
export INTENT_MODEL=BAAI/bge-small-en-v1.5      # macOS / Linux
$env:INTENT_MODEL="BAAI/bge-small-en-v1.5"      # Windows PowerShell
set INTENT_MODEL=BAAI/bge-small-en-v1.5        # Windows Command Prompt
```

Step 0: baseline. Frozen sentence embeddings + logistic regression. This is the number to beat.

Listing 2: `baseline.py`

```
"""Step 0: frozen sentence embeddings + logistic regression. No fine-tuning.
Prints the report and saves it to baseline.txt."""
import json
from pathlib import Path

from sentence_transformers import SentenceTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

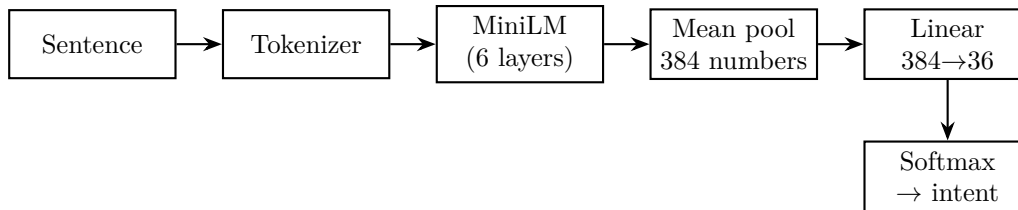
from model import MODEL_NAME

rows = [json.loads(line) for line in open("data/train.jsonl", encoding="utf-8") if line.
        strip()]
texts = [r["text"] for r in rows]
labels = [r["intents"][0] for r in rows]
X_tr, X_te, y_tr, y_te = train_test_split(
    texts, labels, test_size=0.15, stratify=labels, random_state=0)

encoder = SentenceTransformer(MODEL_NAME)
clf = LogisticRegression(max_iter=2000).fit(encoder.encode(X_tr), y_tr)
report = classification_report(y_te, clf.predict(encoder.encode(X_te)), zero_division=0)
report = f"model: {MODEL_NAME}\n\n{report}"
print(report)
Path("baseline.txt").write_text(report, encoding="utf-8")
```

```
python ../../code/baseline.py      # also writes baseline.txt
```

Step 1: fine-tune. Encoder + mean pooling + one linear layer (sizes below are for MiniLM).



Listing 3: model.py

```

"""Encoder + mean pooling + one linear layer (same shape as EVA/Models/singleintentmodel)
.
Pick the encoder with the INTENT_MODEL env var (see MODELS.md); default is MiniLM."""
import os

import torch
from torch import nn
from transformers import AutoModel

MODEL_NAME = os.environ.get("INTENT_MODEL", "sentence-transformers/all-MiniLM-L6-v2")

class IntentModel(nn.Module):
    def __init__(self, num_labels):
        super().__init__()
        self.encoder = AutoModel.from_pretrained(MODEL_NAME)
        self.head = nn.Linear(self.encoder.config.hidden_size, num_labels)

    def forward(self, batch):
        hidden = self.encoder(**batch).last_hidden_state  # (B, tokens, hidden)
        mask = batch["attention_mask"].unsqueeze(-1).float()
        pooled = (hidden * mask).sum(1) / mask.sum(1).clamp(min=1e-9)
        return self.head(pooled)  # (B, num_labels) logits

def best_device():
    if torch.backends.mps.is_available():
        return torch.device("mps")
    if torch.cuda.is_available():
        return torch.device("cuda")
    return torch.device("cpu")

```

train.py holds out 15%, trains 10 epochs, prints accuracy each epoch, then lists every miss. Expect >0.95 and a win over the baseline; if not, fix your data.

Listing 4: train.py

```

"""Step 1: fine-tune the encoder (INTENT_MODEL env var) + linear head on data/train.jsonl
, save to runs/."""
import json
import os
import random

import torch
from sklearn.metrics import accuracy_score, f1_score

```

```

from sklearn.model_selection import train_test_split
from transformers import AutoTokenizer

from model import MODEL_NAME, IntentModel, best_device

EPOCHS, BATCH, LR = 10, 16, 5e-5
random.seed(0)
torch.manual_seed(0)
device = best_device()

rows = [json.loads(line) for line in open("data/train.jsonl", encoding="utf-8") if line.
        strip()]
labels = sorted({r["intents"][0] for r in rows})
label2id = {label: i for i, label in enumerate(labels)}
texts = [r["text"] for r in rows]
ys = [label2id[r["intents"][0]] for r in rows]
X_tr, X_te, y_tr, y_te = train_test_split(
    texts, ys, test_size=0.15, stratify=ys, random_state=0)

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = IntentModel(len(labels)).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=LR)

def encode(batch_texts):
    enc = tokenizer(batch_texts, padding=True, truncation=True,
                    max_length=64, return_tensors="pt")
    return {k: v.to(device) for k, v in enc.items()}

def predict_ids(batch_texts):
    model.eval()
    with torch.no_grad():
        return [i for s in range(0, len(batch_texts), BATCH)
                for i in model(encode(batch_texts[s:s + BATCH])).argmax(1).tolist()]

for epoch in range(1, EPOCHS + 1):
    model.train()
    order = list(range(len(X_tr)))
    random.shuffle(order)
    losses = []
    for s in range(0, len(order), BATCH):
        idx = order[s:s + BATCH]
        logits = model(encode([X_tr[i] for i in idx]))
        target = torch.tensor([y_tr[i] for i in idx], device=device)
        loss = torch.nn.functional.cross_entropy(logits, target)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        losses.append(loss.item())
    pred = predict_ids(X_te)
    print(f"epoch {epoch}  loss {sum(losses) / len(losses):.3f}  "
          f"acc {accuracy_score(y_te, pred):.3f}  ")

```



```

        f"macro-F1 {f1_score(y_te, pred, average='macro'):.3f}")

# Show every mistake on the held-out set: this is where your data needs work.
for text, gold, guess in zip(X_te, y_te, pred):
    if gold != guess:
        print(f"MISS  '{text}'  expected {labels[gold]}  got {labels[guess]}")

os.makedirs("runs", exist_ok=True)
torch.save(model.state_dict(), "runs/model.pt")
json.dump(label2id, open("runs/label2id.json", "w", encoding="utf-8"), indent=2)
json.dump({"model": MODEL_NAME, "accuracy": accuracy_score(y_te, pred),
          "macro_f1": f1_score(y_te, pred, average="macro"),
          "train_rows": len(X_tr), "test_rows": len(X_te),
          "epochs": EPOCHS, "batch": BATCH, "lr": LR},
          open("runs/metrics.json", "w", encoding="utf-8"), indent=2)

```

```
python ../../code/train.py
```

6 Evaluate and predict

Listing 5: predict.py

```
"""Load runs/model.pt and classify text.
python predict.py "open the nav menu"      one sentence
"""

import json
import os
import re
import sys

import torch
from transformers import AutoTokenizer

if not os.path.exists("runs/metrics.json"):
    sys.exit("No runs/metrics.json. Run python ../../code/train.py from your folder first
.")
# Use the encoder train.py actually used, not whatever INTENT_MODEL is set to now.
metrics = json.load(open("runs/metrics.json", encoding="utf-8"))
os.environ["INTENT_MODEL"] = metrics.get("model", "sentence-transformers/all-MiniLM-L6-v2")
sys.path.append(os.path.join("..", "..", "code")) # lets a copy in your folder find
model.py
from model import MODEL_NAME, IntentModel # noqa: E402

MIN_CONF = 0.5 # below this, answer "unhandled" instead of guessing

label2id = json.load(open("runs/label2id.json", encoding="utf-8"))
labels = sorted(label2id, key=label2id.get)
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = IntentModel(len(labels))
model.load_state_dict(torch.load("runs/model.pt", map_location="cpu"))
model.eval()

SLOT_INTENTS = {"Get_coordinates", "Set_navigation_target",
                "Add_waypoint", "Delete_waypoint", "Complete_task"}
# Naive: the slot is whatever follows the last trigger word. Improve this.
TRIGGER = re.compile(r"\b(?:to|named|called|for|of|waypoint|task)\s+", re.I)

def extract_slot(text):
    parts = TRIGGER.split(text.strip().rstrip("?!"))
    return parts[-1].strip() if len(parts) > 1 else None

def predict(text):
    with torch.no_grad():
        probs = model(tokenizer([text], return_tensors="pt")).softmax(1)[0]
        conf, idx = probs.max(0)
        intent = labels[idx.item()] if conf.item() >= MIN_CONF else "unhandled"
        result = {"text": text, "selected_intent": intent, "confidence": round(conf.item(),
3)}
    if intent in SLOT_INTENTS:
```

```
        result["slot"] = extract_slot(text)
    return result

if __name__ == "__main__":
    if len(sys.argv) > 1:
        print(json.dumps(predict(" ".join(sys.argv[1:])))
    else:
        sys.exit(__doc__)
```

```
python ../../code/predict.py "add a waypoint named Olympus Ridge"
{"text": "add a waypoint named Olympus Ridge", "selected_intent": "Add_waypoint",
 "confidence": 0.969, "slot": "Olympus Ridge"}
```

Anything under 0.5 confidence comes back as **unhandled**.

Held-out accuracy is not the real test, because you wrote both halves. The real test: write 10+ new sentences you didn't plan for and see what breaks. Common confusions: primary vs. secondary, scrubber A vs. B, coolant gas vs. liquid, opening the nav menu vs. rerouting, and the coordinate/target/waypoint intents. Fix confusions with better data, not code.

Names: `extract_slot` takes the text after the last trigger word. It's naive ("mark task soil sample complete" → **soil sample complete**). Improving it is a bonus: copy `predict.py` into your own folder, improve it there, and run your copy.

7 Stretch: make it talk

```
pip install faster-whisper sounddevice pyttsx3
python ../../code/voice.py
```

macOS asks for microphone permission for your terminal the first time. Allow it, then restart the terminal. macOS speaks with the built-in `say`; Windows and Linux use `pyttsx3`.

Listing 6: `voice.py`

```
"""Stretch goal: microphone -> Whisper -> intent model -> spoken reply.
pip install faster-whisper sounddevice pyttsx3"""
import subprocess
import sys

import sounddevice as sd
from faster_whisper import WhisperModel

from predict import predict

RATE, SECONDS = 16000, 4
# Domain words help Whisper spell things the way your training data does.
HINT = "EVA, UIA, CO2, O2, scrubber, coolant, waypoint, RPM, HUD"

stt = WhisperModel("base.en", device="cpu", compute_type="int8")

def speak(text):
    if sys.platform == "darwin":
        subprocess.run(["say", text]) # pyttsx3 can hang on repeat calls on macOS
    else:
        import pyttsx3 # Windows (SAPI5) / Linux (espeak)
        engine = pyttsx3.init()
        engine.say(text)
        engine.runAndWait()

while True:
    input("Press Enter, then speak for 4 seconds...")
    audio = sd.rec(RATE * SECONDS, samplerate=RATE, channels=1, dtype="float32")
    sd.wait()
    segments, _ = stt.transcribe(audio.flatten(), language="en", initial_prompt=HINT)
    text = " ".join(s.text for s in segments).strip()
    if not text:
        continue
    result = predict(text)
    print(result)
    if result["selected_intent"] == "unhandled":
        speak("Sorry, say that again.")
    else:
        speak(result["selected_intent"].replace("_", " "))
```

Next steps: `small.en` Whisper, speak the real value back, push-to-talk.

8 What to hand in

One big PR. Commit and push as often as you like, but open a single PR only when everything below is done. No partial PRs. Only touch your own folder, so the seven PRs never conflict.

- ☐ `data/train.jsonl`: 0 problems in `validate.py`
- ☐ `runs/metrics.json`: fine-tuned model beats `baseline.txt`
- ☐ `runs/label2id.json`, `baseline.txt`
- ☐ `notes.md`: model used, results, 10+ unseen sentences, what failed
- ☐ **Not** `runs/model.pt` (`.gitignore` skips it)

The PR is done when every box is checked. Ayush reviews and merges. Stuck for over an hour? Ask in the channel.

Editing this guide: change the `.tex`, rebuild, and commit both the `.tex` and the PDF.

```
latexmk -pdf claws_intent_onboarding.tex
```